

8.1 Introduction and Examples

There are many situations in cryptography where it is important to be able to generate random numbers, bitstrings, etc. For example, cryptographic keys are normally generated at random from a specified key space, and many encryption schemes and signature schemes require random numbers to be generated during their execution. Generating random numbers by means of coin tosses or other physical processes is time-consuming and expensive, so in practice it is common to use a *pseudo-random bit generator*. A bit generator starts with a short random bitstring (a *seed*) and expands it into a much longer bitstring. Thus, a bit generator reduces the number of random bits that are required in a cryptographic application.

More formally, we have the following definition.

Definition 8.1: Let k, ℓ be positive integers such that $\ell \geq k + 1$. A (k, ℓ) -bit generator is a function $f : (\mathbb{Z}_2)^k \rightarrow (\mathbb{Z}_2)^\ell$ that can be computed in polynomial time (as a function of k). The input $s_0 \in (\mathbb{Z}_2)^k$ is called the *seed*, and the output $f(s_0) \in (\mathbb{Z}_2)^\ell$ is called the *generated bitstring*. It will always be required that ℓ is a polynomial function of k .

The function f is deterministic, so the bitstring $f(s_0)$ is dependent only on the seed. Our goal is that the generated bitstring $f(s_0)$ should “look like” truly random bits, given that the seed is chosen at random. If this property holds, then the bit generator will be “secure” and it will be termed a *pseudo-random bit generator* (or *PRBG*). Giving a precise definition of security of a bit generator is quite difficult, but we will try to give an intuitive explanation of the concept as we progress through this chapter.

One specific motivating example for studying pseudo-random bit generators is as follows. Recall the concept of perfect secrecy that we studied in Chapter 2. One realization of perfect secrecy is the *One-time Pad*, where the plaintext and

the key are both bitstrings of a specified length, and the ciphertext is constructed by taking the bitwise exclusive-or of the plaintext and the key. The practical difficulty of the *One-time Pad* is that the key, which must be randomly generated and communicated over a secure channel, must be as long as the plaintext in order to ensure perfect secrecy. Pseudo-random bit generators provide a possible way of alleviating this problem. Suppose Alice and Bob agree on a PRBG and communicate a seed over the secure channel. Alice and Bob can then both compute the same string of pseudo-random bits, which will be xor-ed with the plaintext. Thus the seed functions as a key, and the PRBG can be thought of as a keystream generator for a stream cipher. (Of course, when we replace the one-time pad with a string of pseudo-random bits, we lose the perfect secrecy property.)

Random number generators are used in numerous areas of computer science in addition to cryptography. Applications include simulations, Monte Carlo algorithms, sampling, testing, and many others. Often, it is sufficient for generated random numbers to have a relatively uniform distribution. Various characteristics of a sequence of pseudorandom numbers can be measured, such as frequencies, runs and gaps between the numbers in the sequence. The uniformity of these statistics are often evaluated using traditional tests such as a χ^2 test.

The security of a generator for cryptographic use is more difficult to achieve, however. Passing the above-mentioned tests would certainly be necessary for cryptographic security, but it is not sufficient. Cryptographic security relies on notations such as polynomial-time predictability, which is introduced in Section 8.2.1.

We now present some well-known bit generators to motivate and illustrate some of the concepts we will be studying. First, we observe that a linear feedback shift register, as described in Section 1.1.7, can be thought of as a bit generator. Given a k -bit seed, an LFSR of degree k can be used to produce as many as $2^k - k - 1$ further bits before repeating. The bit generator obtained from an LFSR is very insecure: we already observed in Section 1.2.5 that knowledge of any $2k$ consecutive generated bits suffice to allow the seed to be determined, and hence the entire sequence can then be reconstructed by an opponent. (Although we have not yet defined security of a bit generator, it should be clear that the existence of an attack of this type means that the generator is insecure!)

Another well-known (but insecure) bit generator, called the *Linear Congruential Generator*, is presented as Algorithm 8.1. The basic idea is to generate a sequence of residues modulo M , where each element in the sequence is a certain linear function modulo M of the previous element in the sequence. The seed will be a residue modulo M , and the least significant bits of the elements in the sequence form the generated bitstring.

Here is a very small example to illustrate the workings of the *Linear Congruential Generator*. This example also demonstrates the periodic nature of PRBGs, i.e., that PRBGs eventually repeat if a sufficient number of bits are generated.

Algorithm 8.1: Linear Congruential Generator

Suppose $M \geq 2$ is an integer and suppose $1 \leq a, b \leq M - 1$. Define $k = 1 + \lfloor \log_2 M \rfloor$ and let $k + 1 \leq \ell \leq M - 1$.

The seed is an integer s_0 , where $0 \leq s_0 \leq M - 1$. (Observe that the binary representation of a seed is a bitstring of length not exceeding k ; however, not all bitstrings of length k are permissible seeds.) Now, define

$$s_i = (as_{i-1} + b) \bmod M$$

for $1 \leq i \leq \ell$, and then define

$$f(s_0) = (z_1, z_2, \dots, z_\ell),$$

where

$$z_i = s_i \bmod 2.$$

$1 \leq i \leq \ell$. Then f is a (k, ℓ) -Linear Congruential Generator.

Example 8.1 Suppose we construct a $(5, 10)$ -bit generator by taking $M = 31$, $a = 3$ and $b = 5$ in the *Linear Congruential Generator*. Suppose we consider the mapping $s \mapsto 3s + 5 \bmod 31$. Then $13 \mapsto 13$, and the other 30 residues are permuted in a cycle of length 30, namely

0, 5, 20, 3, 14, 16, 22, 9, 1, 8,
29, 30, 2, 11, 7, 26, 21, 6, 23, 12,
10, 4, 17, 25, 18, 28, 27, 24, 15, 19.

If the seed is anything other than 13, then the seed specifies a starting point in this cycle, and the next 10 elements, reduced modulo 2, form the pseudo-random sequence.

The 31 possible bitstrings produced by this generator are shown in Table 8.1. For example, the sequence constructed from the seed 0 is obtained by taking the ten integers following 0 in the above list, namely, 5, 20, 3, 14, 16, 22, 9, 1, 8, 29, and reducing them modulo 2. $\square$

We can use some concepts developed in earlier chapters to construct bit generators. For example, the output feedback mode of a block cipher, as described in Section 3.7, can be thought of as a bit generator; moreover, it appears to be computationally secure if the underlying block cipher satisfies some reasonable security properties.

Another approach in constructing very fast bit generators is to combine LFSRs

TABLE 8.1
Bitstrings produced by the linear congruential generator

seed	sequence	seed	sequence
0	1010001101	16	0110100110
1	0100110101	17	1001011010
2	1101010001	18	0101101010
3	0001101001	19	0101000110
4	1100101101	20	1000110100
5	0100011010	21	0100011001
6	1000110010	22	1101001101
7	0101000110	23	0001100101
8	1001101010	24	1101010001
9	1010011010	25	0010110101
10	0110010110	26	1010001100
11	1010100011	27	0110101000
12	0011001011	28	1011010100
13	1111111111	29	0011010100
14	0011010011	30	0110101000
15	1010100011		

in some way so that the output looks non-linear. One such method, due to Copersmith, Krawczyk and Mansour, is called the *Shrinking Generator*. Suppose we have two LFSRs, one of degree k_1 and one of degree k_2 . We will require a total of $k_1 + k_2$ bits as our seed, in order to initialize both LFSRs. The first LFSR will produce a sequence of bits, say $a_1, a_2, \dots$, and the second one produces a sequence of bits $b_1, b_2, \dots$. Then we define a sequence of bits $z_1, z_2, \dots$ by the rule

$$z_i = a_{i_k},$$

where i_k is the position of the k th "1" in the sequence $b_1, b_2, \dots$. These bits comprise a subsequence of the bits generated by the first LFSR.

In the rest of this chapter, we will investigate bit generators that can be proved to be secure given some plausible computational assumption. There are PRBGs based on the fundamental problems of factoring (as it relates to the RSA Cryptosystem) and the **Discrete Logarithm** problem. A PRBG based on the RSA encryption function is shown in Algorithm 8.2, and a PRBG based on the **Discrete Logarithm** problem is discussed in the Exercises. Briefly, the *RSA Generator* chooses an initial element of $\mathbb{Z}_n^*$ to be the seed. A sequence of elements of $\mathbb{Z}_n^*$ is formed, in which each element in the sequence is the RSA encryption of the previous element. Then the least significant bits of the elements in the sequence form the generated bitstring.

We now give an example of the *RSA Generator*.

Example 8.2 Suppose $n = 91261 = 263 \times 347$, $b = 1547$, and $s_0 = 75634$.

Algorithm 8.2: RSA Generator

Suppose p, q are two $(k/2)$ -bit primes, and define $n = pq$. Suppose b is chosen such that $\gcd(b, \phi(n)) = 1$. As always, n and b are public while p and q are secret.

A seed s_0 is any element of $\mathbb{Z}_n^*$, so s_0 has k bits. For $i \geq 1$, define

$$s_{i+1} = s_i^b \bmod n,$$

and then define

$$f(s_0) = (z_1, z_2, \dots, z_\ell),$$

where

$$z_i = s_i \bmod 2,$$

$1 \leq i \leq \ell$. Then f is a (k, ℓ) -RSA Generator.

The first 20 bits produced by the *RSA Generator* are computed as shown in Table 8.2. The bitstring resulting from this seed is

10000111011110011000.

8.2 Indistinguishability of Probability Distributions

There are two main objectives of a pseudo-random number generator: it should be fast (i.e., computable in polynomial time as a function of k) and it should be secure. Of course, these two requirements are often conflicting. The bit generators based on linear congruences or linear feedback shift registers are indeed very fast. These bit generators are quite useful in simulations, but they are very insecure for cryptographic applications.

Let us now try to make precise the idea of what it means for a bit generator to be "secure." Intuitively, a string of ℓ bits produced by a bit generator should look "random." That is, it should be impossible in an amount of time that is polynomial in k (equivalently, polynomial in ℓ) to distinguish a string of ℓ bits produced by a PRBG from a string of ℓ truly random bits.

For example, if a bit generator produced "1"s with probability $2/3$, say, then it would usually be easy to distinguish a generated bitstring from a truly random

TABLE 8.2
Bits produced by the RSA Generator

i	s_i	z_i
0	75634	
1	31483	1
2	31238	0
3	51968	0
4	39796	0
5	28716	0
6	14089	1
7	5923	1
8	44891	1
9	62284	0
10	11889	1
11	43467	1
12	71215	1
13	10401	1
14	77444	0
15	56794	0
16	78147	1
17	72137	1
18	89592	0
19	29022	0
20	13356	0

bitstring. To be specific, we might use the following distinguishing strategy. Suppose we were given a bitstring of length ℓ . Denote the number of "1"s in this bitstring by ℓ_1 . On average, a random bitstring of length ℓ will contain $\ell/2$ "1"s, and a generated bitstring will contain (on average) $2\ell/3$ "1"s. Therefore, if

$$\ell_1 > \frac{\ell/2 + 2\ell/3}{2} = \frac{7\ell}{12},$$

then we would guess that this bitstring was more likely to be a generated bitstring than a random bitstring.

This example motivates the idea of distinguishability of probability distributions. We will now give a definition of this concept. In this definition and elsewhere, we will denote an ℓ -tuple $(z_1, \dots, z_\ell)$ by z^ℓ for ease of notation.

Definition 8.2: Suppose p_0 and p_1 are two probability distributions on the set $(\mathbb{Z}_2)^\ell$ of all bitstrings of length ℓ . For $j = 0, 1$ and $z^\ell \in (\mathbb{Z}_2)^\ell$, $p_j(z^\ell)$ denotes the probability of the string z^ℓ occurring in the distribution p_j . Let $\text{dst} : (\mathbb{Z}_2)^\ell \rightarrow \{0, 1\}$ be a function and let $\epsilon > 0$. For $j = 0, 1$, define

$$E_{\text{dst}}(p_j) = \sum_{\{z^\ell \in (\mathbb{Z}_2)^\ell : \text{dst}(z^\ell)=1\}} p_j(z^\ell).$$

We say that dst is an ϵ -distinguisher of p_0 and p_1 provided that

$$|E_{\text{dst}}(p_0) - E_{\text{dst}}(p_1)| \geq \epsilon,$$

and we say that p_0 and p_1 are ϵ -distinguishable if there exists an ϵ -distinguisher of p_0 and p_1 .

A distinguisher, say dst , is a *polynomial-time distinguisher* provided that $\text{dst}(z^\ell)$ can be computed in polynomial time as a function of ℓ .

The intuition behind the definition of a distinguisher is as follows. The function (or algorithm) dst tries to decide if a given bitstring z^ℓ of length ℓ is more likely to have arisen from probability distribution p_0 or from probability distribution p_1 . The output $\text{dst}(z^\ell)$ represents the distinguisher's guess as to which of these two probability distributions is more likely to have produced z^ℓ . The quantity $E_{\text{dst}}(p_j)$ represents the average (i.e., expected) value of the output of dst over the probability distribution p_j , for $j = 0, 1$. Then dst is an ϵ -distinguisher provided that the values of these two expectations are at least ϵ apart.

REMARK In the definition above, a distinguisher is a function, i.e., a deterministic algorithm. We could generalize the definition so that a distinguisher could be a randomized algorithm, if desired. That is, given an ℓ -tuple $z^\ell = (z_1, \dots, z_\ell)$, a distinguisher might guess "0" with some probability p (depending on z^ℓ), and hence guess "1" with probability $1 - p$. In the case of a randomized distinguisher, it is not hard to see that

$$E_{\text{dst}}(p_j) = \sum_{z^\ell \in (\mathbb{Z}_2)^\ell} (p_j(z^\ell) \times \Pr[\text{dst}(z^\ell) = 1]).$$

All of the results we state and prove remain true for randomized distinguishers.

The relevance of distinguishers to PRBGs is as follows. Consider the sequence of ℓ bits produced by a bit generator. There are 2^ℓ possible sequences, and if these ℓ bits were chosen independently and uniformly at random, then each of these 2^ℓ sequences would occur with equal probability $1/2^\ell$. Thus a truly random sequence corresponds to an equiprobable (or uniform) distribution on the set of

all bitstrings of length ℓ . Suppose we denote this uniform probability distribution by p_u .

Now, consider sequences produced by the bit generator f . Suppose a k -bit seed is chosen uniformly at random and the generator is used to construct a bitstring of length ℓ . Then we obtain a probability distribution on the set of all bitstrings of length ℓ , which we denote by p_f . For the purposes of illustration, suppose we make the simplifying assumption that no two seeds give rise to the same sequence of bits. Then, of the 2^ℓ possible sequences, 2^k sequences each occur with probability $1/2^k$, and the remaining $2^\ell - 2^k$ sequences never occur. Hence, the probability distribution p_f is very non-uniform.

Even though the two probability distributions p_u and p_f may be quite different, it is still conceivable that they might be ϵ -distinguishable in polynomial time only for small values of ϵ . This is our objective in constructing PRBGs. However, this goal can be quite difficult to achieve. As the next example shows, producing 0's and 1's with equal probability is not sufficient to ensure indistinguishability.

Example 8.3 Suppose that a bit generator f only produces sequences in which exactly $\ell/2$ bits have the value 0 and $\ell/2$ bits have the value 1. Define the function dst by

$$\text{dst}(z_1, \dots, z_\ell) = \begin{cases} 1 & \text{if } (z_1, \dots, z_\ell) \text{ has exactly } \ell/2 \text{ bits equal to 0} \\ 0 & \text{otherwise.} \end{cases}$$

It is not hard to see that

$$E_{\text{dst}}(p_u) = \frac{\binom{\ell}{\ell/2}}{2^\ell}$$

and

$$E_{\text{dst}}(p_f) = 1.$$

It can be shown that

$$\lim_{\ell \rightarrow \infty} \frac{\binom{\ell}{\ell/2}}{2^\ell} = 0.$$

Hence, for any fixed value of $\epsilon < 1$, p_u and p_f are ϵ -distinguishable if ℓ is sufficiently large. $\square$

8.2.1 Next Bit Predictors

Another useful concept in studying bit generators is that of a *next bit predictor*, which works as follows. Let f be a (k, ℓ) -bit generator. Suppose $1 \leq i \leq \ell - 1$, and we have a function $\text{nbp} : (\mathbb{Z}_2)^{i-1} \rightarrow \mathbb{Z}_2$, which takes as input an $(i-1)$ -tuple $z^{i-1} = (z_1, \dots, z_{i-1})$, which represents the first $i-1$ bits produced by f (given an unknown, random, k -bit seed). Then the function nbp attempts to predict the next bit produced by f , namely, z_i . We say that the function nbp is an

ϵ -*ith* bit predictor if nbp can predict the i th bit of the generated bitstring (given the first $i-1$ bits) with probability at least $1/2 + \epsilon$, where $\epsilon > 0$.

We can give a more precise formulation of this concept in terms of probability distributions, as follows. We have already defined a probability distribution p_f on $(\mathbb{Z}_2)^\ell$ induced by the bit generator f . We can also look at the probability distributions induced by f on any one of the ℓ generated bits (or indeed on any subsequence of these ℓ generated bits). So, for $1 \leq i \leq \ell$, we will think of the i th generated bit as a random variable that we will denote by z_i .

In view of these definitions, we have the following characterization of an i th bit predictor.

THEOREM 8.1 Let f be a (k, ℓ) -bit generator. Then the function nbp is an ϵ -*ith* bit predictor for f if and only if

$$\sum_{z^{i-1} \in (\mathbb{Z}_2)^{i-1}} (p_f(z^{i-1}) \times \Pr[z_i = \text{nbp}(z^{i-1}) | z^{i-1}]) \geq \frac{1}{2} + \epsilon.$$

PROOF The probability of correctly predicting the i th generated bit is computed by summing (over all possible $(i-1)$ -tuples $z^{i-1} = (z_1, \dots, z_{i-1})$) the product of the probability that the $(i-1)$ -tuple z^{i-1} is produced by the bit generator f and the probability that the i th bit is predicted correctly, given the $(i-1)$ -tuple z^{i-1} . $\square$

REMARK The reason for the expression $1/2 + \epsilon$ in this definition is that any predicting algorithm will predict any bit of a truly random bitstring with probability $1/2$. If a generated bitstring is not truly random, then it may be possible to predict a given bit with higher probability. (Note that it is unnecessary to consider functions that predict a given bit with probability less than $1/2$, because in this case a function that replaces every prediction z by $1 - z$ will predict the bit with probability greater than $1/2$.)

Also, we could allow bit predictors to be randomized algorithms. The results we state and prove will remain true in this generalized setting. In fact, in one of the main results in this chapter (Theorem 8.3), we will be constructing a bit predictor that is randomized. $\square$

We illustrate these ideas by producing a next-bit predictor for the *Linear Congruential Generator* of Example 8.1.

Example 8.1 (Cont.) For any i such that $1 \leq i \leq 9$, define an i th bit predictor by the formula

$$\text{nbp}(z^{i-1}) = 1 - z_{i-1}.$$

That is, the function nbp predicts that a 0 is most likely to be followed by a 1, and vice versa. It is not hard to compute from Table 8.1 that nbp is a $\frac{9}{62}$ -next

Algorithm 8.3: DISTINGUISH(z^i)

```

external nbp
 $z \leftarrow \text{nbp}(z^{i-1})$ 
if  $z = z_i$ 
  then return (1)
else return (0)

```

bit predictor for any $i \geq 1$ (i.e., it predicts any bit correctly with probability $1/2 + 9/62 = 20/31$). $\square$

We can use a next bit predictor to construct a distinguishing algorithm. Suppose that **nbp** is an ϵ -ith bit predictor for a given integer $i \leq \ell$. The distinguisher is constructed as shown in Algorithm 8.3.

The input to the algorithm DISTINGUISH is a sequence of i bits, denoted by z^i . DISTINGUISH uses the function **nbp** to predict z_i , given the first $i-1$ bits in the sequence. If the predicted value is the same as the actual value of z_i , then the algorithm DISTINGUISH outputs "1," meaning that it deems the i -tuple z^i to have arisen from the bit generator f . Otherwise, DISTINGUISH outputs "0."

It is clear that DISTINGUISH (Algorithm 8.3) is a polynomial-time Turing reduction. The next theorem shows that DISTINGUISH is a good distinguisher provided that **nbp** is a good i th bit predictor.

THEOREM 8.2 Suppose **nbp** is a (polynomial-time) ϵ -ith bit predictor for the (k, ℓ) -bit generator f . Let p_f be the probability distribution induced on $(\mathbb{Z}_2)^\ell$ by f , and let p_u be the uniform probability distribution on $(\mathbb{Z}_2)^\ell$. Then Algorithm 8.3 is a (polynomial-time) ϵ -distinguisher of p_f and p_u .

PROOF First, observe that

$$\text{DISTINGUISH}(z^i) = 1 \Leftrightarrow \text{nbp}(z^{i-1}) = z_i.$$

Thus, we can compute the expectation $E_{\text{DISTINGUISH}}(p_f)$ as follows:

$$\begin{aligned} E_{\text{DISTINGUISH}}(p_f) &= \sum_{z^i \in (\mathbb{Z}_2)^i} (p_f(z^i) \times \Pr[\text{DISTINGUISH}(z^i) = 1]) \\ &= \sum_{z^i \in (\mathbb{Z}_2)^i} (p_f(z^i) \times \Pr[\text{nbp}(z^{i-1}) = z_i]). \end{aligned}$$

Now, consider any $(i-1)$ -tuple z^{i-1} . Define

$$z = (z_1, \dots, z_{i-1}, 0)$$

and

$$z' = (z_1, \dots, z_{i-1}, 1).$$

Consider the two terms in the above sum corresponding to z and z' . Then we have

$$\begin{aligned} p_f(z) \times \Pr[\text{nbp}(z^{i-1}) = 0] &+ p_f(z') \times \Pr[\text{nbp}(z^{i-1}) = 1] \\ &= p_f(z^{i-1}) \times \sum_{j \in \{0,1\}} (\Pr[z_i = j | z^{i-1}] \times \Pr[\text{nbp}(z^{i-1}) = j]) \\ &= p_f(z^{i-1}) \times \Pr[z_i = \text{nbp}(z^{i-1}) | z^{i-1}]. \end{aligned}$$

It therefore follows that

$$\begin{aligned} E_{\text{DISTINGUISH}}(p_f) &= \sum_{z^{i-1} \in (\mathbb{Z}_2)^{i-1}} (p_f(z^{i-1}) \times \Pr[z_i = \text{nbp}(z^{i-1}) | z^{i-1}]) \\ &\geq \frac{1}{2} + \epsilon, \end{aligned}$$

because **nbp** is an ϵ -ith bit predictor.

On the other hand, any predictor will predict the i th bit of a truly random sequence with probability $1/2$. Therefore, it is easy to see that $E_{\text{DISTINGUISH}}(p_u) = 1/2$. Hence,

$$|E_{\text{DISTINGUISH}}(p_u) - E_{\text{DISTINGUISH}}(p_f)| \geq \epsilon,$$

as desired. $\square$

One of the main results in the theory of pseudo-random bit generators, due to Yao, is that a next bit predictor is a *universal test*. That is, a bit generator is "secure" if and only if there does not exist any polynomial-time ϵ -ith bit predictor for the generator, except for very small values of ϵ . Theorem 8.2 proves the implication in one direction. To prove the converse, we need to show how the existence of a distinguisher implies the existence of a certain i th bit predictor. This is done in Theorem 8.3.

THEOREM 8.3 Suppose **dst** is a (polynomial-time) ϵ -distinguisher of p_f and p_u , where p_f is the probability distribution induced on $(\mathbb{Z}_2)^\ell$ by the (k, ℓ) -bit generator f , and p_u is the uniform probability distribution on $(\mathbb{Z}_2)^\ell$. Then for some i , $1 \leq i \leq \ell-1$, there exists a (polynomial-time) $\frac{\epsilon}{4}$ -ith bit predictor for f .

PROOF For $0 \leq i \leq \ell$, define q_i to be the probability distribution on $(\mathbb{Z}_2)^\ell$ where the first i bits are generated using the bit generator f (assuming that the k -bit seed is chosen uniformly at random), and the remaining $\ell-i$ bits are generated uniformly and independently at random. Thus $q_0 = p_u$ and $q_\ell = p_f$. We are given that

$$|E_{\text{dst}}(q_0) - E_{\text{dst}}(q_\ell)| \geq \epsilon.$$

Algorithm 8.4: NBP(z^{i-1})

external dst
 choose $(z_1, \dots, z_\ell) \in (\mathbb{Z}_2)^{\ell-i+1}$ at random
 $z \leftarrow \text{dst}(z_1, \dots, z_\ell)$
return $(z + z_i \bmod 2)$

Applying the triangle inequality, we have that

$$\epsilon \leq |E_{\text{dst}}(q_0) - E_{\text{dst}}(q_\ell)| \leq \sum_{i=1}^{\ell} |E_{\text{dst}}(q_{i-1}) - E_{\text{dst}}(q_i)|.$$

Hence, it follows that there is at least one value i , $1 \leq i \leq \ell$, such that

$$|E_{\text{dst}}(q_{i-1}) - E_{\text{dst}}(q_i)| \geq \frac{\epsilon}{\ell}.$$

We will assume that

$$E_{\text{dst}}(q_{i-1}) - E_{\text{dst}}(q_i) \geq \frac{\epsilon}{\ell}$$

(alternatively, if

$$E_{\text{dst}}(q_i) - E_{\text{dst}}(q_{i-1}) \geq \frac{\epsilon}{\ell},$$

then straightforward modifications to our proof will yield the desired result).

We are going to construct an i th bit predictor (for this specified value of i). The predicting algorithm is a randomized algorithm and it is presented as Algorithm 8.4. It is clear that Algorithm 8.4 is a polynomial-time (randomized) Turing reduction.

Here is the idea behind Algorithm 8.4. NBP in fact produces an ℓ -tuple according to the probability distribution q_{i-1} , given that $z_1, \dots, z_{i-1}$ are generated by the bit generator f . If dst answers "0," then it thinks that the ℓ -tuple was most likely generated according to the probability distribution q_i . Now q_{i-1} and q_i differ only in that the i th bit is generated at random in the distribution q_{i-1} , whereas it is generated using the bit generator f in the distribution q_i . Hence, when dst answers "0," it thinks that the i th bit, z_i , is what would be produced by the bit generator, f . Hence, in this case we take z_i as our prediction of the i th bit. On the other hand, if dst answers "1," it thinks that z_i is random, so we take $1 - z_i$ as our prediction of what the i th bit would be under the generator f .

We need to compute the probability that the i th bit is predicted correctly. Observe that if dst answers "0," then the prediction is correct with probability

$$\Pr[z_i = z_i | z^{i-1}].$$

If dst answers "1," then the prediction is correct with probability

$$\Pr[z_i \neq z_i | z^{i-1}].$$

In our computation, we will make use of a useful fact that relates the probability distribution q_{i-1} to the probability distribution q_i . Namely, the following holds:

$$q_{i-1}(z^i) \times \Pr[z_i = z_i | z^{i-1}] = \frac{q_i(z^i)}{2}. \quad (8.1)$$

This can be proven fairly easily, as follows:

$$\begin{aligned} q_{i-1}(z^i) \times \Pr[z_i = z_i | z^{i-1}] &= q_{i-1}(z^{i-1}) \times \frac{1}{2^{\ell-i+1}} \times \Pr[z_i = z_i | z^{i-1}] \\ &= q_i(z^i) \times \frac{1}{2^{\ell-i+1}} \\ &= \frac{q_i(z^i)}{2}. \end{aligned}$$

Now we are ready to perform our main computation, which consists of a complicated series of sums of probabilities. The reader is encouraged to work through the proof, filling in all the details required to proceed from each line to the next. It is useful to note that the fact (8.1) proven above will be used in a couple of places later on in the proof.

$$\begin{aligned} & \sum_{z^{i-1} \in (\mathbb{Z}_2)^{i-1}} (p_f(z^{i-1}) \times \Pr[z_i = \text{nbp}(z^{i-1})]) \\ &= \sum_{z^i \in (\mathbb{Z}_2)^i} (q_{i-1}(z^i) \times \Pr[z_i = \text{nbp}(z^{i-1})]) \\ &= \sum_{z^i \in (\mathbb{Z}_2)^i} (q_{i-1}(z^i) (\Pr[\text{dst}(z^i) = 0] \times \Pr[z_i = z_i | z^{i-1}] + \\ & \quad \Pr[\text{dst}(z^i) = 1 | z^i] \times \Pr[z_i \neq z_i | z^{i-1}])) \\ &= \sum_{z^i \in (\mathbb{Z}_2)^i} (q_{i-1}(z^i) ((1 - \Pr[\text{dst}(z^i) = 1 | z^i]) \times \Pr[z_i = z_i | z^{i-1}] + \\ & \quad \Pr[\text{dst}(z^i) = 1 | z^i] \times (1 - \Pr[z_i = z_i | z^{i-1}])))) \\ &= \sum_{z^i \in (\mathbb{Z}_2)^i} (q_{i-1}(z^i) \times \Pr[z_i = z_i | z^{i-1}]) \\ & \quad - 2 \sum_{z^i \in (\mathbb{Z}_2)^i} (q_{i-1}(z^i) \times \Pr[\text{dst}(z^i) = 1 | z^i] \times \Pr[z_i = z_i | z^{i-1}]) \\ & \quad + \sum_{z^i \in (\mathbb{Z}_2)^i} (q_{i-1}(z^i) \times \Pr[\text{dst}(z^i) = 1 | z^i]) \end{aligned}$$

Algorithm 8.5: Blum-Blum-Shub Generator

Let p, q be two $(k/2)$ -bit primes such that $p \equiv q \equiv 3 \pmod{4}$, and define $n = pq$. Let $\mathbf{QR}(n)$ denote the set of quadratic residues modulo n .

A seed s_0 is any element of $\mathbf{QR}(n)$. For $0 \leq i \leq \ell - 1$, define

$$s_{i+1} = s_i^2 \pmod{n},$$

and then define

$$f(s_0) = (z_1, z_2, \dots, z_\ell),$$

where

$$z_i = s_i \pmod{2}.$$

$1 \leq i \leq \ell$. Then f is a (k, ℓ) -PRBG, called the *Blum-Blum-Shub Generator*, which we abbreviate to *BBS Generator*.

One way to choose an appropriate seed is to select an element $s_{-1} \in \mathbb{Z}_n^*$ and compute $s_0 = s_{-1}^2 \pmod{n}$. This ensures that $s_0 \in \mathbf{QR}(n)$.

$$\begin{aligned} &= \frac{1}{2} \sum_{z^i \in \mathbb{Z}_2^{\ell}} q_i(z^i) - \sum_{z^i \in \mathbb{Z}_2^{\ell}} (q_i(z^i) \times \Pr[\mathbf{dst}(z^i) = 1 | z^i]) \\ &\quad + \sum_{z^i \in \mathbb{Z}_2^{\ell}} (q_{i-1}(z^i) \times \Pr[\mathbf{dst}(z^i) = 1 | z^i]) \\ &= \frac{1}{2} + E_{\mathbf{dst}}(q_{i-1}) - E_{\mathbf{dst}}(q_i) \\ &\geq \frac{1}{2} + \frac{\epsilon}{\ell}, \end{aligned}$$

which was what we wanted to prove. □

8.3 The Blum-Blum-Shub Generator

In this section we will describe and analyze one of the most popular PRBGs, due to Blum, Blum, and Shub. For any odd integer n , denote the quadratic residues modulo n by $\mathbf{QR}(n)$. That is, $\mathbf{QR}(n) = \{x^2 \pmod{n} : x \in \mathbb{Z}_n^*\}$. The *Blum-Blum-Shub Generator* is presented in Algorithm 8.5.

The generator works quite simply. Given a seed $s_0 \in \mathbf{QR}(n)$, we compute the sequence $s_1, s_2, \dots, s_\ell$ by successive squarings modulo n , and then we reduce

TABLE 8.3
Bits produced by BBS generator

i	s_i	z_i
0	20749	
1	143135	1
2	177671	1
3	97048	0
4	89992	0
5	174051	1
6	80649	1
7	45663	1
8	69442	0
9	186394	0
10	177046	0
11	137922	0
12	123175	1
13	8630	0
14	114386	0
15	14863	1
16	133015	1
17	106065	1
18	45870	0
19	137171	1
20	48060	0

each s_i modulo 2 to obtain z_i . It follows that

$$z_i = (s_0^{2^i} \pmod{n}) \pmod{2}.$$

$1 \leq i \leq \ell$.

We now give a toy example of the *BBS Generator*.

Example 8.4 Suppose $n = 192649 = 383 \times 503$ and $s_0 = 101355^2 \pmod{n} = 20749$. The first 20 bits produced by the *BBS Generator* are computed as shown in Table 8.3. Hence the bitstring resulting from this seed is

11001110000100111010.

Next, we review some results on Jacobi symbols from Section 5.4 and some other number-theoretic facts from other parts of Chapter 5. Suppose p and q are two distinct odd primes, and let $n = pq$. From the definition of the Jacobi symbol,

it is easily seen that

$$\left(\frac{x}{n}\right) = \begin{cases} 0 & \text{if } \gcd(x, n) > 1 \\ 1 & \text{if } \left(\frac{x}{p}\right) = \left(\frac{x}{q}\right) = 1 \text{ or if } \left(\frac{x}{p}\right) = \left(\frac{x}{q}\right) = -1 \\ -1 & \text{if one of } \left(\frac{x}{p}\right) \text{ and } \left(\frac{x}{q}\right) = 1 \text{ and the other} = -1. \end{cases}$$

Recall that x is a quadratic residue modulo n if and only if

$$\left(\frac{x}{p}\right) = \left(\frac{x}{q}\right) = 1.$$

Define

$$\widetilde{\mathbf{QR}}(n) = \left\{ x \in \mathbb{Z}_n^* \setminus \mathbf{QR}(n) : \left(\frac{x}{n}\right) = 1 \right\}.$$

Thus

$$\widetilde{\mathbf{QR}}(n) = \left\{ x \in \mathbb{Z}_n^* : \left(\frac{x}{p}\right) = \left(\frac{x}{q}\right) = -1 \right\}.$$

An element $x \in \widetilde{\mathbf{QR}}(n)$ is called a *pseudo-square* modulo n . It is not hard to see that $|\mathbf{QR}(n)| = |\widetilde{\mathbf{QR}}(n)| = (p-1)(q-1)/4$.

The security of the *Blum-Blum-Shub Generator* is based on the intractability of the **Composite Quadratic Residues** problem, which we define as Problem 8.1. (In Chapter 5, we defined the **Quadratic Residues** problem modulo a prime and showed that it is easy to solve; here we have a composite modulus.)

Problem 8.1: Composite Quadratic Residues

Instance: A positive integer n that is the product of two unknown distinct odd primes p and q , and an integer $x \in \mathbb{Z}_n^*$ such that $\left(\frac{x}{n}\right) = 1$.

Question: Is $x \in \mathbf{QR}(n)$?

Basically, the **Composite Quadratic Residues** problem requires us to distinguish quadratic residues modulo n from pseudo-squares modulo n . This can be no more difficult than factoring n . For if the factorization $n = pq$ can be computed, then it is a simple matter to calculate $\left(\frac{x}{p}\right)$, say. Given that $\left(\frac{x}{n}\right) = 1$, it follows that x is a quadratic residue modulo n if and only if $\left(\frac{x}{p}\right) = 1$.

There does not seem to be any way to solve the **Composite Quadratic Residues** problem efficiently if the factorization of n is not known. So it is commonly conjectured that this problem is intractable if it is infeasible to factor n .

Here is a property of the *BBS Generator* that is important when we look at its security. Since $n = pq$ where $p \equiv q \equiv 3 \pmod{4}$, it follows that, for any quadratic residue x , there is a unique square root of x that is also a quadratic residue. This particular square root is called the *principal square root* of x . As a consequence, the mapping $x \rightarrow x^2 \pmod{n}$, which is used to define the *BBS Generator*, is a permutation on $\mathbf{QR}(n)$, the set of quadratic residues modulo n .

Example 8.5 Suppose that $n = 253 = 11 \times 23$. Then

$$|\mathbf{QR}(n)| = \frac{10 \times 22}{4} = 55.$$

It can be shown that *BBS Generator* in $\mathbb{Z}_{55}$ permutes the elements of $|\mathbf{QR}(55)|$ in one cycle of length 1, one cycle of length 4, one cycle of length 10 and two cycles of length 20. $\square$

8.3.1 Security of the BBS Generator

In this section, we look at the security of the *BBS Generator* in detail. We begin by supposing that the pseudo-random bits produced by the *BBS Generator* are ϵ -distinguishable from ℓ random bits, and then see where that leads us. We will present a series of polynomial-time Turing reductions. Throughout this section, $n = pq$, where p and q are distinct primes such that $p \equiv q \equiv 3 \pmod{4}$, and the factorization $n = pq$ is unknown.

We have already discussed the idea of a next bit predictor. In this section, we consider a similar concept that we call a *previous bit predictor*. A previous bit predictor for a (k, ℓ) -*BBS Generator* will take as input ℓ pseudo-random bits produced by the generator (as determined by an unknown random seed $s_0 \in \mathbf{QR}(n)$), and attempt to compute the value $z_0 = s_0 \pmod{2}$. A previous bit predictor can be a probabilistic algorithm, and we say that a previous bit predictor **pbp** is an ϵ -previous bit predictor if its probability of correctly guessing z_0 is at least $1/2 + \epsilon$, where this probability is computed over all possible seeds s_0 .

We state the following theorem, which is similar to Theorem 8.3, without proof.

THEOREM 8.4 Suppose there exists a (polynomial-time) ϵ -distinguisher of p_f and p_u , where p_f is the probability distribution induced on $(\mathbb{Z}_2)^\ell$ by the (k, ℓ) -*BBS Generator*, f , and p_u is the uniform probability distribution on $(\mathbb{Z}_2)^\ell$. Then there exists a (polynomial-time) $\frac{\epsilon}{4}$ -previous bit predictor for f .

Next, we show how to use a δ -previous bit predictor, **pbp**, to construct a probabilistic algorithm that distinguishes quadratic residues modulo n from pseudo-squares modulo n with probability $1/2 + \delta$. The algorithm **QR-TEST**, which is presented as Algorithm 8.6, uses **pbp** as a subroutine, or oracle. It is assumed that the input, x , is an element with Jacobi symbol equal to 1, i.e., $x \in \mathbf{QR}(n) \cup \widetilde{\mathbf{QR}}(n)$.

Briefly, Algorithm 8.6 constructs a sequence of ℓ bits by first squaring x and then using x^2 as a seed in the *BBS Generator*. If $x \in \mathbf{QR}(n)$, then x itself is a "legal" seed and the resulting sequence is identical to the one which would be obtained by running the *BBS Generator* with seed x . If $x \in \widetilde{\mathbf{QR}}(n)$, then the sequence is what would be obtained by running the *BBS Generator* with seed $-x$. The algorithm tests whether $x \in \mathbf{QR}(n)$ by appealing to the predictor **pbp**, as described in the following theorem and its proof.

Algorithm 8.6: QR-TEST(x, n)

```

external pbp
 $s_1 \leftarrow x^2 \bmod n$ 
comment:  $s_1$  is a quadratic residue modulo  $n$ 

 $z_1 \leftarrow s_1 \bmod 2$ 
use the BBS Generator to compute  $z_2, \dots, z_\ell$  from seed  $s_1$ 
 $z \leftarrow \text{pbp}(z_1, \dots, z_\ell)$ 
if  $(x \bmod 2) = z$ 
  then return (yes)
else return (no)

```

THEOREM 8.5 Suppose pbp is a (polynomial-time) δ -previous bit predictor for the (k, ℓ) -BBS Generator f . Then the algorithm QR-TEST, determines quadratic residuosity correctly (in polynomial time) with probability at least $1/2 + \delta$, where this probability is averaged over all possible inputs $x \in \text{QR}(n) \cup \overline{\text{QR}}(n)$, which are chosen uniformly at random.

PROOF Since $n = pq$ and $p \equiv q \equiv 3 \pmod{4}$, it follows that $(\frac{-1}{n}) = 1$, so $-1 \in \overline{\text{QR}}(n)$. Let s_0 denote the principal square root of s_1 . We are assuming that $(\frac{x}{n}) = 1$. Therefore, $s_0 = x$ if $x \in \text{QR}(n)$; whereas $s_0 = -x$ if $x \in \overline{\text{QR}}(n)$.

The ℓ bits $z_1, \dots, z_\ell$ are exactly those that would be generated by using the BBS Generator with s_0 as the seed. Now, the integer n is odd, so

$$(-x \bmod n) \bmod 2 \neq (x \bmod n) \bmod 2.$$

We have that

$$x \in \text{QR}(n) \Leftrightarrow s_0 = x.$$

Therefore, it follows that QR-TEST gives the correct answer if and only if pbp correctly predicts z . The desired result then follows immediately. $\square$

Example 8.6 Suppose that $n = 60485929729$ and $x = 349850938$. It can be verified that $(\frac{x}{n}) = 1$. QR-TEST will first compute $s_1 = 41061588913$ and $z_1 = 1$. Then we generate $\ell - 1$ additional bits using the BBS Generator. The sequence $s_1, s_2, \dots$ begins with

41061588913, 24724816839, 40968882391, 2137662714, 32677932305.

Hence, the first five generated bits are

1, 1, 1, 0, 1.

Algorithm 8.7: MC-QR-TEST(x)

```

external QR-TEST
choose  $r \in \mathbb{Z}_n^*$  randomly
 $x' \leftarrow r^2 x \bmod n$ 
choose  $s \in \{1, -1\}$  randomly
 $x' \leftarrow s x' \bmod n$ 
 $t \leftarrow \text{QR-TEST}(x')$ 
if  $((t = \text{yes}) \text{ and } (s = 1)) \text{ or } ((t = \text{no}) \text{ and } (s = -1))$ 
  then return (yes)
else return (no)

```

The sequence of bits can be continued as far as desired.

We know that the seed for this sequence is either x or $-x$, depending on whether or not $x \in \text{QR}(n)$. The constructed bit sequence is given to the predictor, which returns a value $z = 0$ or 1. Suppose that the value $z = 0$ is returned by pbp . Since $x \bmod 2 = 0 = z$, QR-TEST outputs "yes," meaning that it believes that $x \in \text{QR}(n)$. $\square$

Theorem 8.5 shows how we can distinguish pseudo-squares from quadratic residues with probability at least $1/2 + \delta$. However, this success probability is an average over all possible inputs $x \in \text{QR}(n) \cup \overline{\text{QR}}(n)$. We now show that this result can be improved to yield a Monte Carlo algorithm that determines quadratic residuosity correctly with probability at least $1/2 + \delta$. In other words, for any $x \in \text{QR}(n) \cup \overline{\text{QR}}(n)$, the Monte Carlo algorithm we will describe gives the correct answer with probability at least $1/2 + \delta$. Note that this algorithm is an unbiased algorithm (i.e., it may give an incorrect answer for any input), in contrast to the Monte Carlo algorithms that we studied in Section 5.4 which were all biased algorithms.

The Monte Carlo algorithm MC-QR-TEST is presented in Algorithm 8.7. It calls the previous algorithm QR-TEST as a subroutine. Basically, Algorithm 8.7 is a randomization process. It constructs a residue x' by multiplying x by a random quadratic residue, and then randomly multiplying the result by ± 1 . The value x' is tested for quadratic residuosity using the subroutine QR-TEST, and this information is used to make a decision on the quadratic residuosity of x .

THEOREM 8.6 Suppose that QR-TEST determines quadratic residuosity correctly (in polynomial time) with average probability at least $1/2 + \delta$. Then the algorithm MC-QR-TEST, which is described in Algorithm 8.7, is a (polynomial-time) Monte Carlo algorithm for Composite Quadratic Residues having error

probability at most $1/2 - \delta$.

PROOF For any given input $x \in \mathbf{QR}(n) \cup \widetilde{\mathbf{QR}}(n)$, Algorithm 8.7 produces an element x' that is a random element of $\mathbf{QR}(n) \cup \widetilde{\mathbf{QR}}(n)$. Moreover, the status of x as a quadratic residue can be determined from the status of x' as a quadratic residue. $\square$

The last algorithm in our series of reductions is a general result pertaining to unbiased Monte Carlo algorithms. It shows that any (unbiased) Monte Carlo algorithm having error probability at most $1/2 - \delta$ can be used to construct an unbiased Monte Carlo algorithm with error probability at most γ , for any $\gamma > 0$. In other words, we can make the probability of correctness arbitrarily close to 1. The idea is to run the given Monte Carlo algorithm $2m + 1$ times, for some integer m , and take the "majority vote" as the answer. By computing the error probability of this algorithm, we can also see how m depends on γ . This dependence is stated in the following theorem.

THEOREM 8.7 Suppose A is an unbiased Monte Carlo algorithm with error probability at most $1/2 - \delta$. Suppose we define an algorithm A^n by running A $n = 2m + 1$ times on a given instance I , and outputting the most frequent answer. Then the error probability of the algorithm A^n is at most

$$\frac{(1 - 4\delta^2)^m}{2}.$$

PROOF The probability of obtaining exactly i correct answers in the n trials is at most

$$\binom{n}{i} \left(\frac{1}{2} - \delta\right)^i \left(\frac{1}{2} + \delta\right)^{n-i}.$$

The probability that the most frequently occurring answer is incorrect is equal to the probability that the number of correct answers in the n trials is at most m .

Hence, we compute as follows:

$$\begin{aligned} \Pr[\text{error}] &\leq \sum_{i=0}^m \binom{n}{i} \left(\frac{1}{2} + \delta\right)^i \left(\frac{1}{2} - \delta\right)^{2m+1-i} \\ &= \left(\frac{1}{2} + \delta\right)^m \left(\frac{1}{2} - \delta\right)^{m+1} \sum_{i=0}^m \binom{n}{i} \left(\frac{1/2 - \delta}{1/2 + \delta}\right)^{m-i} \\ &\leq \left(\frac{1}{2} + \delta\right)^n \left(\frac{1}{2} - \delta\right)^{m+1} \sum_{i=0}^m \binom{n}{i} \\ &= \left(\frac{1}{2} + \delta\right)^m \left(\frac{1}{2} - \delta\right)^{m+1} 2^{2m} \\ &= \left(\frac{1}{4} - \delta^2\right)^m \left(\frac{1}{2} - \delta\right) 2^{2m} \\ &= (1 - 4\delta^2)^m \left(\frac{1}{2} - \delta\right) \\ &\leq \frac{(1 - 4\delta^2)^m}{2}, \end{aligned}$$

as required. $\square$

Suppose we want to lower the probability of error to some value γ , where $0 < \gamma < 1/2 - \delta$. We need to choose m so that

$$\frac{(1 - 4\delta^2)^m}{2} \leq \gamma.$$

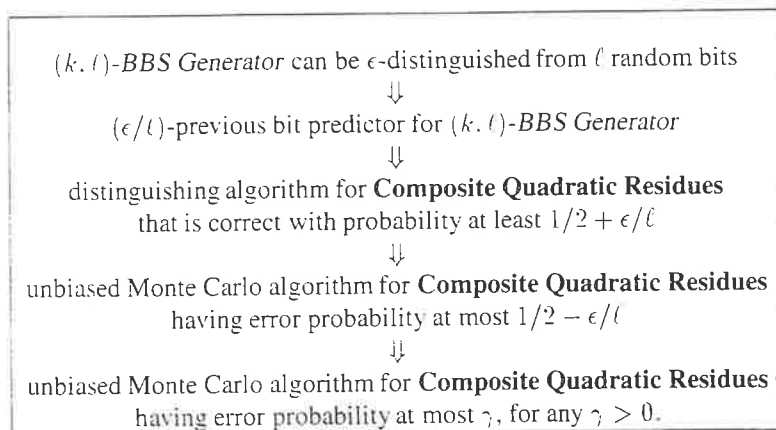
Hence, it suffices to take

$$m = \left\lceil \frac{1 + \log_2 \gamma}{\log_2(1 - 4\delta^2)} \right\rceil.$$

If algorithm A is run $2m + 1$ times, then the majority vote yields the correct answer with probability at least $1 - \gamma$. It is not hard to show that this value of m is at most $c/(1 - \delta^2)$ for some constant c . Hence, the number of times that the algorithm must be run is polynomial in $1/\gamma$ and $1/\delta$.

Example 8.7 Suppose we start with a Monte Carlo algorithm that gives the correct answer with probability at least .55, so $\delta = .05$. If we desire a Monte Carlo algorithm in which the probability of error is at most .05, then it suffices to take $m = 230$, so $n = 461$. $\square$

Let us combine all the reductions we have done. We have the following sequence of implications:



The reductions are all polynomial-time algorithms (i.e., polynomial time as a function of k , $1/\epsilon$, and $1/\gamma$). Since it is widely believed that there is no polynomial-time Monte Carlo algorithm for **Composite Quadratic Residues** with small error probability, we have some evidence that the *BBS Generator* is secure. This is yet another example of provable security.

We close this section by mentioning a way of improving the efficiency of the *BBS Generator*. The sequence of pseudo-random bits is constructed by taking the least significant bit of each s_i , where $s_i = s_0^{2^i} \bmod n$. Suppose instead that we extract the r least significant bits from each s_i , for some positive integer r . This will improve the efficiency of the PRBG by a factor of r , but we need to ask if the PRBG will remain secure (assuming the intractability of **Composite Quadratic Residues**). It has been shown that this approach is secure provided that $r \leq \log_2 \log_2 n$. So we can extract about $\log_2 \log_2 n$ pseudo-random bits per modular squaring. In a realistic implementation of the *BBS Generator*, say one in which $n \approx 10^{160}$, we can extract nine bits per squaring operation.

8.4 Probabilistic Encryption

In Section 5.9.2, we discussed notions such as semantic security of cryptosystems and ciphertext indistinguishability. An example of a semantically secure public-key cryptosystem was provided, based on a secure family of trapdoor one-way permutations and some appropriate hash functions. A proof of security was given, which applies when the hash functions are modeled as random oracles. The concrete system known as *Optimal Asymmetric Encryption Padding* is based on this type of construction.

In this section, we describe an alternative approach, known as probabilistic encryption, which is an interesting idea of Goldwasser and Micali. We begin with a definition of this concept. The definition is based on distinguishability of probability distributions.

Definition 8.3: A probabilistic public-key cryptosystem is defined to be a six-tuple $(\mathcal{P}, \mathcal{C}, \mathcal{K}, \mathcal{E}, \mathcal{D}, \mathcal{R})$, where $\mathcal{P}$ is the set of plaintexts, $\mathcal{C}$ is the set of ciphertexts, $\mathcal{K}$ is the keyspace, $\mathcal{R}$ is a set of randomizers, and for each key $K \in \mathcal{K}$, $e_K \in \mathcal{E}$ is a public encryption rule and $d_K \in \mathcal{D}$ is a secret decryption rule. The following properties should be satisfied:

1. Each $e_K : \mathcal{P} \times \mathcal{R} \rightarrow \mathcal{C}$ and $d_K : \mathcal{C} \rightarrow \mathcal{P}$ are functions such that

$$d_K(e_K(b, r)) = b$$

for every plaintext $b \in \mathcal{P}$ and every $r \in \mathcal{R}$. (In particular, this implies that $e_K(x, r) \neq e_K(x', r)$ if $x \neq x'$.)

2. Security of the scheme is defined as follows. Let ϵ be a specified security parameter. For any fixed $K \in \mathcal{K}$ and for any $x \in \mathcal{P}$, define a probability distribution $p_{K,x}$ on $\mathcal{C}$, where $p_{K,x}(y)$ denotes the probability that y is the ciphertext given that K is the key and x is the plaintext (this probability is computed over all random choices $r \in \mathcal{R}$). Suppose $x, x' \in \mathcal{P}$, $x \neq x'$, and $K \in \mathcal{K}$. Then the probability distributions $p_{K,x}$ and $p_{K,x'}$ are not ϵ -distinguishable in polynomial time.

Here is how the scheme works. To encrypt a plaintext x , choose a (random) randomizer $r \in \mathcal{R}$ and compute $y = e_K(x, r)$. By property 1, any ciphertext $y = e_K(x, r)$ can be decrypted uniquely to the plaintext x . Property 2 is stating that the probability distribution of all encryptions of x cannot be distinguished (in polynomial time) from the probability distribution of all encryptions of x' if $x' \neq x$. Informally, an encryption of x "looks like" an encryption of x' . The security parameter ϵ should be small: in practice we would want to have $\epsilon = c/|\mathcal{R}|$ for some small $c > 0$.

It should be clear that a probabilistic public-key cryptosystem (as defined above) provides semantic security. Property 2 is stating that ciphertexts encrypting any two given plaintexts should be indistinguishable in polynomial time. This is the same requirement we had in Section 5.9.2.

We now present the *Goldwasser-Micali Public-key Cryptosystem* as Cryptosystem 8.1. Note that this cryptosystem encrypts every bit of plaintext independently, i.e., the plaintext space is $\mathcal{P} = \{0, 1\}$.

As noted above, this system encrypts one bit at a time. A 0 bit is encrypted to a random quadratic residue modulo n ; a 1 bit is encrypted to a random pseudo-square modulo n . When Bob receives an element $y \in \mathcal{QR}(n) \cup \mathcal{QR}(n)$, he can use his knowledge of the factorization of n to determine whether $y \in \mathcal{QR}(n)$ or